TÀI LIỆU KỸ THUẬT SỐ 03 / BỘ 04 TÀI LIỆU TRIỂN KHAI

PHƯƠNG PHÁP LUẬN & CƠ CHẾ CÁC KIẾN TRÚC CANDIDATE

Giải thích nguyên lý hoạt động, lý do đề xuất và bản đồ ghép nối các tổ hợp thay thế hệ thống MARS-SQL 21B

> **Tóm tắt tư tưởng thiết kế:** Hệ thống MARS-SQL gốc dùng 3 mô hình lớn giống hệt nhau (3 × 7B = 21 tỷ tham số) cho cả 3 việc: Lọc bảng, Viết SQL và Chấm điểm. Cách làm này giống như "dùng đại bác để bắn chim" ở khâu lọc bảng và chấm điểm, gây tốn 41–47 GB VRAM và mất 22.12 giây/câu hỏi. Kiến trúc đề xuất thay thế mỗi khâu bằng đúng công cụ chuyên biệt nhẹ nhất (kết hợp Toán đồ thị + Mô hình nhỏ 0.5B–3B + Khóa văn phạm + Bộ chấm điểm 0-token), giúp cắt giảm 82%–87% tham số mà còn sửa được các lỗi cố hữu của bản 21B.

1. Bốn Điểm Nghẽn Của MARS-SQL 21B & Lý Do Phải Thay Thế

Trong bài báo gốc của **Yang et al. (2026)**, MARS-SQL đạt 77.84% độ chính xác trên tập BIRD-dev và 89.75% trên Spider-test, nhưng gặp 4 rào cản lớn khi đưa vào ứng dụng thực tế:

1. **Quá nặng và quá chậm (21B tham số):** Nạp đồng thời ba mô hình `Qwen2.5-Coder-7B` (mỗi bản nặng 15.22 GB BF16) ngốn từ **41 đến 47 GB VRAM**, tiêu tốn trung bình **13,325 token** và mất **22.12 giây** cho mỗi câu hỏi.
2. **Bị "mù cấu trúc" ở Agent 1 (Mất bảng cầu nối ở 48 câu hỏi):** Agent 1 (7B) kiểm tra từng bảng một cách rời rạc xem có từ vựng giống câu hỏi hay không. Do đó, những "bảng cầu nối" trung gian (bridge tables — không chứa từ khóa trong câu hỏi nhưng bắt buộc phải có để nối khóa ngoại `JOIN` giữa hai bảng chính) bị vứt bỏ oan uổng, làm hỏng hoàn toàn 48 câu hỏi trên tập BIRD-dev. Thêm vào đó, trên tập dữ liệu sạch như Spider-test, việc tắt hẳn Agent 1 7B chỉ làm giảm 0.56% độ chính xác (từ 89.75% xuống 89.19%), cho thấy dùng mô hình 7B để lọc bảng là sự lãng phí lớn.
3. **Ảo giác tên bảng và lỗi cú pháp ở Agent 2:** Agent 2 (7B) viết câu lệnh SQL tự do không có thanh chắn văn phạm nên thường xuyên bịa tên bảng (ví dụ gõ nhầm `fprm` thay vì bảng thật `frpm`), làm tốn nhiều vòng lặp gọi cơ sở dữ liệu chỉ để sửa những lỗi chính tả cơ bản.
4. **Điểm chấm cảm tính chưa hiệu chuẩn ở Agent 3:** Agent 3 (7B) đọc lại 8 phương án rồi lấy xác suất sinh ra chữ `"Yes"` làm điểm tin cậy. Con số xác suất này chưa được hiệu chuẩn (uncalibrated), không chỉ ra được câu lệnh vi phạm quy tắc nào và lại ngốn thêm một mô hình 7B thứ ba.

2. Giải Phẫu Giai Đoạn 1: Các Candidate Thay Thế Grounding Agent

Candidate 1A: LitE-SQL (Qwen3-Embedding-0.6B + HN-SupCon) Piao et al. (2026)

- **Lý do đề xuất:** Thay thế mô hình 7B nặng 15.22 GB bằng một mô hình nhúng vector chỉ **0.6B tham số** (nhỏ hơn 11 lần), loại bỏ hoàn toàn việc bắt LLM phải đọc đi đọc lại cấu trúc bảng ở mỗi câu hỏi.
- **Cơ chế hoạt động dễ hiểu:** Toàn bộ thông tin của từng cột trong cơ sở dữ liệu được biến thành các vector toán học và lưu sẵn trên ổ cứng từ trước (offline). Khi người dùng đặt câu hỏi, hệ thống chỉ cần biến câu hỏi đó thành 1 vector rồi tìm nhanh 25 cột có vector gần nhất (cosine similarity). Nhờ được huấn luyện bằng hàm tương phản lọc mẫu âm khó (HN-SupCon — dạy phân biệt các cột trùng tên như `id`, `name` ở các bảng khác nhau), LitE-SQL tìm trúng cột cần thiết tới **95.23%** trên tập BIRD.

Candidate 1B: Graph-Link (Cây Steiner NWST + Relation-Aware GNN) Zhong et al. (2026)

- **Lý do đề xuất:** Trị tận gốc căn bệnh "quên bảng cầu nối" ở 48 câu hỏi BIRD-dev mà cả MARS-SQL 21B lẫn LitE-SQL đứng một mình đều mắc phải.
- **Cơ chế hoạt động dễ hiểu:** Hãy hình dung cơ sở dữ liệu như một **bản đồ giao thông**, trong đó các bảng/cột là các bến xe và các liên kết khóa ngoại (PK-FK) là đường quốc lộ nối giữa chúng. Sau khi LitE-SQL (và mạng đồ thị GNN) chọn ra các cột quan trọng nhất khớp với câu hỏi làm "điểm phải đến", thuật toán toán học **Cây Steiner có trọng số nút (Node-Weighted Steiner Tree)** sẽ tự động tìm mạng lưới đường đi ngắn nhất nối liền tất cả các điểm đó lại. Nhờ vậy, bất kỳ bảng cầu nối nào nằm ở ngã tư trung gian đều được tự động giữ lại 100%, giúp độ chính xác trên các câu hỏi khó (cần nối nhiều bảng) tăng vọt thêm **+13.8%**.

Candidate 1C: NanoJev-0.6B Parallel Column Scorer TianyuCodings (2026)

- **Lý do đề xuất:** Dành cho cấu hình tiết kiệm VRAM tối đa (Combo B), cho phép dùng chung duy nhất một mô hình `NanoJev-0.6B` cho cả việc chọn cột ở Agent 1 lẫn việc chấm điểm câu SQL ở Agent 3.
- **Cơ chế hoạt động dễ hiểu:** Thay vì sinh chữ, NanoJev-0.6B đưa toàn bộ danh sách cột vào đầu quyết định Đúng/Sai (Boolean/Choice Head) và chấm điểm tất cả các cột cùng một lúc trong 1 lượt chạy duy nhất (0-token decoding) trước khi chuyển cho Cây Steiner nối bảng.

3. Giải Phẫu Giai Đoạn 2: Các Candidate Thay Thế Generation Agent

Candidate 2A, 2B & 2C: Các Mô Hình Ngôn Ngữ Nhỏ Chuyên Viết SQL (0.5B – 3B)

- **FINER-SQL-3B Hoang et al. (2026):** Mô hình 3B tham số (chỉ nặng 1.93 GB khi nén Q4 hoặc 6.18 GB ở F16) được học từ ngân hàng lời giải mẫu (Reasoning Bank) và huấn luyện tăng cường với 4 đầu điểm chi tiết. Dù nhỏ hơn một nửa, bản 3B này đạt **67.73% EX** trên BIRD-dev (thời gian chạy 5.57 giây), đánh bại cả mô hình 7B Generator-Only của MARS-SQL (66.37%) và CodeS-15B (58.47%).

- **SLM-SQL (0.5B – 1.5B) & CSC-SQL Sheng & Xu (2025a):** Dành cho máy tính cá nhân dưới 8 GB RAM hoặc chạy thẳng trên trình duyệt web (bản 0.5B chỉ nặng 1.4 GB). Bí quyết giúp bản `SLM-SQL-1.5B` đạt tới **67.08% EX** trên BIRD-dev (vượt OmniSQL-7B 66.10%) là cơ chế **Tự nhất quán có sửa lỗi (Corrective Self-Consistency - CSC)**: mô hình chính viết 16 phương án và gom nhóm theo kết quả chạy thử; nếu các nhóm cho kết quả bất đồng nhau, một mô hình siêu nhỏ thứ hai (0.5B) sẽ nhảy vào đọc các chỗ khác biệt để gộp và sửa lại thành 1 câu lệnh đúng.
- **CodeS (1B & 3B) Li et al. (2024):** Họ mô hình mã nguồn mở được tiền huấn luyện chuyên sâu trên kho mã lệnh SQL với cửa sổ ngữ cảnh 8,192 token, rất dễ tinh chỉnh 1 lượt cho bản Demo.

Candidate 2D: LLM-JEPA — Căn Chỉnh Tư Duy Text-to-Code Huang et al. (2026)

- **Lý do đề xuất:** Các mô hình nhỏ thường chỉ học cách đoán từ tiếp theo chứ không thực sự hiểu rằng "câu hỏi tiếng người" và "câu lệnh SQL" là hai cách diễn đạt của cùng một ý. LLM-JEPA giúp nối thẳng tư duy từ câu hỏi sang cấu trúc code bên trong não bộ mô hình mà **không làm tốn thêm bất kỳ mili-giây hay bộ nhớ nào lúc chạy thực tế (0% inference overhead)**.
- **Cơ chế hoạt động dễ hiểu:** Chỉ áp dụng trong lúc huấn luyện. Hệ thống đặt cả Câu hỏi (gắn thêm token đặc biệt `[PRED]`) và Câu lệnh SQL chuẩn vào chung một lần đọc nhưng dùng tấm chắn (mặt nạ chú ý 2 khối) không cho chúng nhìn thấy nhau. Mô hình vừa phải tập viết từng chữ SQL, vừa phải dùng trạng thái não bộ tại vị trí `[PRED]` của khối Câu hỏi để đoán trúng trạng thái não bộ tổng thể của khối Câu lệnh SQL. Bài tập kép này giúp tăng **+7.8%** độ chính xác trên tập Spider.

Candidate 2E: XGrammar — Khóa Văn Phạm Bằng Ôtômat Đẩy Xuống (PDA) Dong et al. (2024)

- **Lý do đề xuất:** Chặn đứng 100% lỗi sai cú pháp SQL, lỗi gõ nhầm tên bảng/cột (như `fprm` vs `frpm`) và lỗi ép sai kiểu dữ liệu (như viết chuỗi `"4.98"` thay vì số `4.98`) ngay trong lúc mô hình đang gõ từng từ, với tốc độ biên dịch nhanh hơn công cụ cũ Outlines từ **250 đến 1,000 lần**.
- **Cơ chế hoạt động dễ hiểu:** Các công cụ khóa văn phạm cũ (như Outlines) mất tới 2.0–4.9 giây để dịch luật bảng trước khi chạy. XGrammar thông minh hơn nhờ chia từ điển của mô hình thành 2 nhóm: **hơn 99% số từ** là cố định không phụ thuộc ngữ cảnh sâu nên được tính sẵn thành bảng mặt nạ bit lưu trong bộ nhớ đệm; chỉ có **dưới 1% số từ** cần kiểm tra trực tiếp với ngăn xếp (stack) lúc chạy. Nhờ vậy, XGrammar chỉ mất **4–8 mili-giây** để nạp lược đồ bảng và khóa cứng không cho mô hình sinh ra bất kỳ từ nào sai luật SQL hay nằm ngoài danh sách bảng của Agent 1.
- **Lưu ý về hiện tượng "Giải cứu rỗng" (Hollow Rescue) Chavan (2026):** Nghiên cứu thực nghiệm cho thấy XGrammar giúp các mô hình nhỏ 0.6B–4B đạt **100% đúng cấu trúc**, nhưng nếu mô hình nhỏ hiểu sai ý câu hỏi (ví dụ câu hỏi bảo tính trung bình mà mô hình lại viết hàm tính tổng) thì câu lệnh tuy chạy trơn tru vẫn ra đáp số sai. Đó chính là lý do XGrammar bắt buộc phải đi cặp với **LLM-JEPA** (để hiểu đúng ý ở Agent 2) và **PV-SQL** (để bắt lỗi logic ở Agent 3).

4. Giải Phẫu Giai Đoạn 3: Các Candidate Thay Thế Validation Agent

Candidate 3A: PV-SQL — Thăm Dò Dữ Liệu Thật & 10 Quy Tắc Cây Cú Pháp Tian & Zhang (2026)

- **Lý do đề xuất:** Giải quyết 41% số ca làm sai do mô hình không biết dữ liệu thực tế lưu bên trong bảng viết như thế nào, đồng thời bắt lỗi thiếu điều kiện bằng quy tắc Python siêu nhẹ (tốn 0 GB VRAM).
- **Cơ chế hoạt động dễ hiểu:** PV-SQL gồm 2 bước: (1) **Thăm dò (Probing):** Cho phép chạy tối đa 5 câu lệnh ngắn có gắn `LIMIT` để ngó trước vài dòng dữ liệu thật trong bảng (biết ngay cột bang lưu chữ `'CA'` hay `'California'`) trước khi viết câu chốt; (2) **Kiểm tra 10 quy tắc AST:** Phân tích câu lệnh SQL thành cây cú pháp và đối chiếu với 10 quy tắc bắt buộc suy ra từ câu hỏi (Distinct, Top-K, Average, Extreme, Temporal, Compare...). Nếu câu hỏi có chữ "trung bình" mà cây cú pháp thiếu hàm `AVG()`, PV-SQL bác bỏ ngay lập tức và báo lỗi cụ thể để sửa luôn, giúp điểm hiệu quả thực thi VES tăng **+20.8%** (đạt 86.9).

Candidate 3B: Các Mô Hình Quyết Định Nhanh System-1 Giải Mã 0-Token ConvAI (2026)

- **Lý do đề xuất:** Thay thế hoàn toàn mô hình 7B chấm điểm chậm chạp bằng các bộ phân loại siêu nhẹ chấm điểm toàn bộ 8 phương án SQL cùng lúc chỉ trong **1.2 đến 130 mili-giây** mà không cần sinh ra bất kỳ chữ nào (0-token decoding), giảm chi phí tính toán từ 40 đến 400 lần.
- **Bốn ứng viên System-1 tùy chọn theo phần cứng:**
  - **Laya-421M (Khuyên dùng cho Combo C & D):** Dựa trên `ModernBERT-large`, nặng chỉ **~85 MB**, chèn token `[MASK]` cho từng phương án SQL để chấm điểm song song trong **~1.2 ms** (810 quyết định/giây trên GPU T4). Nhờ huấn luyện bằng RLCD, điểm xác suất của Laya cực kỳ chuẩn xác (sai số hiệu chuẩn `ECE = 0.081`, tốt hơn nhiều so với Jev 0.246).
  - **NanoJev-0.6B (Dùng cho Combo B):** Dựa trên `Qwen3-0.6B` gắn 3 đầu quyết định (Choice Head chọn trong 2–255 phương án, Boolean Head và Score Head). Sau khi tinh chỉnh trên dữ liệu SQL, NanoJev chọn đúng quỹ đạo tới **93.8%** trong ~100 ms.
  - **Kev-0.8B (Chuyên gia chống đoán mò):** Đạt độ chính xác **98.0%** và có khả năng chủ động từ chối phán quyết nếu phát hiện cả 8 phương án đều sai hoặc thiếu thông tin.
  - **Von-395M (Chuyên gia chạy CPU cho Combo A):** Bộ mã hóa quyết định tối ưu hóa riêng để chạy mượt trên CPU của laptop không có card đồ họa rời.
- **Cơ chế Cổng dừng sớm (Adaptive Early-Exit Gate):** Ngay khi Agent 2 vừa viết xong câu lệnh đầu tiên, nếu câu đó vượt qua 10 quy tắc của PV-SQL và được Laya-421M chấm độ tự tin vượt mức `0.85`, hệ thống dừng lại và trả kết quả ngay cho người dùng mà không cần chạy thêm vòng lặp hay sinh đủ 8 phương án.

5. Bảng Tổ Hợp 4 Cấu Hình Toàn Tuyến (Combo A, B, C, D)

Tổ Hợp (Combo)	Agent 1 (Grounding)	Agent 2 (Generation)	Agent 3 (Validation) & Phần Cứng
Combo A Siêu Nhẹ (~2.5B)	LitE-SQL (0.6B) + Cây Steiner (Graph-Link)	SLM-SQL-1.5B + LLM-JEPA + XGrammar PDA	PV-SQL + Von-395M (CPU) / Laya-421M. *Chạy máy cá nhân <8 GB RAM, không cần GPU.*
Combo B Dùng Chung (~3.6B)	NanoJev-0.6B (dùng chung) + Cây Steiner	FINER-SQL-3B + LLM-JEPA + XGrammar PDA	PV-SQL + NanoJev-0.6B (Domain SFT). *Chỉ tốn 3.2 GB (Q4) – 7.5 GB (F16) VRAM.*
Combo C Tối Ưu SOTA (~4.0B)	LitE-SQL (0.6B) + Cây Steiner + GNN	FINER-SQL-3B + LLM-JEPA + XGrammar (N≤4, M≤3)	PV-SQL + Laya-421M (RLCD, ~85 MB). *Tối ưu nhất: 3.5–8.5 GB VRAM, ~4.2–5.8s/câu.*
Combo D Độ Chính Xác Cao (~8.0B)	LitE-SQL (0.6B) + Cây Steiner + GNN	Qwen2.5-Coder-7B + LLM-JEPA + XGrammar PDA	PV-SQL + Laya-421M (RLCD). *Chạy vừa 1 card GPU 24 GB, nhắm vượt mốc 77.84% EX.*

Tài Liệu Tham Khảo

1. Yang, H., et al. (2026). *MARS-SQL: Multi-Agent Reinforcement Learning for Text-to-SQL*. ICML 2026 (arXiv:2511.01008).
2. Piao, S., Lee, J., & Park, S. (2026). *LitE-SQL: A Lightweight and Efficient Text-to-SQL Framework with Vector-based Schema Linking and Execution-Guided Self-Correction*. Findings of EACL 2026 (arXiv:2510.09014).
3. Zhong, J., et al. (2026). *Graph-Link: Bridging the Semantic-Structural Gap in Text-to-SQL via Constrained Subgraph Induction*. ICML 2026.
4. TianyuCodings (2026). *NanoJev: A 0.6B Open-Source Parallel Decision Model with Zero Output-Token Decoding*. GitHub Repository.
5. Hoang, T. D., et al. (2026). *FINER-SQL: Boosting Small Language Models for Text-to-SQL with Fine-Grained Execution Feedback and Cost-Efficient Rewards*. IEEE ICDE 2026 (arXiv:2605.03465).
6. Sheng, L., & Xu, S.-S. (2025a). *SLM-SQL: An Exploration of Small Language Models for Text-to-SQL*. Preprint (arXiv:2507.22478).
7. Li, H., et al. (2024). *CodeS: Towards Building Open-source Language Models for Text-to-SQL*. ACM SIGMOD 2024 (arXiv:2402.16347).
8. Huang, H., LeCun, Y., & Balestriero, R. (2026). *LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures*. ICLR 2026 (arXiv:2509.14252).
9. Dong, Y., et al. (2024). *XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models*. CMU / NVIDIA (arXiv:2411.15100).
10. Chavan, A. (2026). *Constrained Decoding Eliminates Structural Failures in Small LLMs but Reveals a Scale-Dependent Semantic Gap*. Preprint (arXiv:2609.23742).
11. Tian, Y., & Zhang, T. (2026). *PV-SQL: Synergizing Database Probing and Rule-based Verification for Text-to-SQL Agents*. Findings of ACL 2026 (arXiv:2604.17653).
12. ConvAI Benchmark Authors (2026). *Laya, CLEF, Kev, and Open System-One Decision Models: ModernBERT Cross-Encoder with RLCD*. Technical Benchmark Reports.